

*Zespół Szkół Technicznych*

*im. Ignacego Mościckiego w Tarnowie*

# *Podstawy PHP*



## **Tworzenie i zastosowanie sesji**

## 1. Funkcje haszujące

Funkcja **skrót**, funkcja **mieszająca** lub funkcja **haszująca** – funkcja przyporządkowująca dowolnie dużej liczbie krótką, zawsze posiadającą stały rozmiar, niespecyficzną, quasi-losową wartość, tzw. skrót nieodwracalny [Wikipedia].

Wynika z tego, że nie istnieją funkcje, które pozwalają na odwrócenie procesu, tj. otrzymania pierwotnej informacji z istniejącego hashu, ale w Internecie znajdują się bazy danych, które zawierają gigantyczne ilości przeliczonych haszy (tzw. tablice tęczowe).

## 2. Popularne funkcje haszujące w PHP

### a) md5()

Składnia:

```
string md5(string $text)
```

Warning: It is not recommended to use this function to secure passwords.

### b) sha1()

Składnia:

```
string sha1(string $text)
```

Warning: It is not recommended to use this function to secure passwords.

### c) crypt()

```
string crypt(string $text, string $salt)
```

Sól – jest to losowa wartość doklejana do każdego hasła przed wykonaniem samego haszowania. Może być statyczna (taka sama dla wszystkich) lub dynamiczna (każde hasło ma inną sól).

Przykład:

```
1  <?php
2      $password = "abcd";
3
4      $pass1=md5($password);
5      $pass2=sha1($password);
6
7      $salt="@#%";
8      $pass3=crypt($password,$salt);
9      echo "<p>md5: $pass1</p>";
10     echo "<p>sha1: $pass2</p>";
11     echo "<p>crypt: $pass3</p>";
12  ?>
```

Wynik na stronie:

md5: e2fc714c4727ee9395f324cd2e7f331f

sha1: 81fe8bfe87576c3ecb22426f8e57847382917acf

crypt: @#rUM4jIw1WwY

#### d) hash()

Składnia:

```
string hash(string $algorithm, string $text)
```

Listę dostępnych algorytmów otrzymujemy za pomocą funkcji hash\_algos():

```
array hash_algos()
```

Przykład:

```
1  <?php
2      $password = "abcd";
3
4      echo "<h2> Lista algorytmów</h2>";
5      $algorithms=hash_algos();
6      echo "<p>";
7      foreach($algorithms as $alg)
8  ▼  {
9          echo "$alg; ";
10     }
11     echo "</p>";
12
13     echo "<h2> Wynik hashowania</h2>";
14     $nr=1;
15     foreach($algorithms as $alg)
16  ▼  {
17         $wynik = hash($alg,$password);
18         echo "<p>$nr. $alg:<br> $wynik </p>";
19         $nr++;
20     }
21
22  ?>
```

### 3. Wprowadzenie do sesji

- **Sesja to tymczasowy zbiór zmiennych istniejący tylko do momentu zamknięcia przeglądarki.**
- Zmienne są przechowywane po stronie serwera, a u klienta trzymane jest tylko ID sesji. To ID jest zapisane w cookie lub przekazywane przez URL.
- Wszystkie zmienne związane z danym użytkownikiem są dostępne za pośrednictwem tablicy superglobalnej **`$_SESSION[]`**.
- Sesja jest aktywna do chwili, gdy użytkownik zamknie przeglądarkę, a cookie zostanie usunięte.
- Jeśli chcielibyśmy dowiedzieć się, ile wynosi czas automatycznego wylogowania, możemy go wyświetlić:

```
echo ini_get('session.gc_maxlifetime');
```

#### 4. Zarządzanie sesją

a) Aby wykorzystać sesję, należy najpierw ją zainicjalizować (uruchomić).

Służy do tego funkcja `session_start()`;

#### b) Identyfikator sesji

Identyfikator sesji można poznać dzięki funkcji `session_id()`. Pozwala ona zarówno pobrać, jak i zmienić identyfikator sesji.

```
string session_id([string $id])
```

#### c) Nazwa sesji

Funkcja `session_name()` pozwala pobrać lub ustawić nazwę sesji.

```
string session_name([string $name])
```

#### Przykład:

```
<?php
    session_start();
    echo "ID:".session_id()."<br>";
    echo "Name: ".session_name()."<br>";
    echo $_COOKIE[session_name()];
?>
```

#### d) Zamykanie sesji

Funkcja `session_destroy()` kończy sesję i usuwa wszystkie zmienne przechowywane w sesji.

#### 5. Zmienne sesyjne

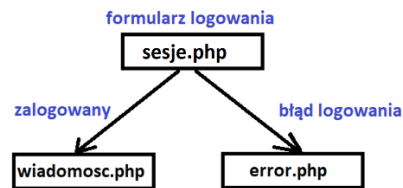
Po otwarciu sesji, w tablicy superglobalnej `$_SESSION[]` można zapisać dowolne dane związane z sesją. Będą one dostępne dla wszystkich innych skryptów korzystających z sesji

```
<?php
    session_start();
    echo "<p>Twój identyfikator sesji to: ".session_id()."</p>";

    $_SESSION['imie'] = "Jan";
    $_SESSION['nazwisko'] = "Kowalski";
    echo "Dane zostały zapisane.";
?>
```

## 6. Uwierzytelnienie za pomocą sesji

Ćwiczenie - schemat witryny:



Założmy, w pliku **sesje.php** znajduje się formularz do logowania.

Po poprawnym zalogowaniu zostanie odczytana wiadomość z pliku **wiadomosc.php**, w przeciwnym razie wyświetli się komunikat błędu zapisany w pliku **sesje\_error.php**

### I. Tworzymy na stronie formularz do logowania

```
<form action="sesje.php" method="post">
  <label>Login: </label>
  <input type="text" name="login">    <br>
  <label>Password: </label>
  <input type="password" name="password">    <br>
  <input type="submit" value="Zaloguj się">
</form>
```

### II. Uruchamiamy sesję w pliku sesje.php

```
1 <?php
2     session_start();
3     ?>
4     <!DOCTYPE html>
5     <html lang="pl">
```

### III. Sprawdzenie pola w tablicy \$\_SESSION[]

Tworzymy w pliku sessje.php dowolne pole w tablicy \$\_SESSION[], np. zalogowany i sprawdzamy, czy ma ustawioną określoną wartość.

```
<?php
    session_start();
    if(isset($_SESSION['zalogowany']) && $_SESSION['zalogowany']=='yes')
    {
        // Przekieruj na stronę z wiadomością
    }
    else
    {
        // Sprawdź dane logowania
    }
?>
```

#### IV. Przekierowanie na stronę z wiadomością – funkcja header

---

```
if(isset($_SESSION['zalogowany']) && $_SESSION['zalogowany']=='yes')
{
    header('location:wiadomosc.php');
}
```

#### V. Sprawdzamy, czy wprowadzono jakieś dane do formularza

---

```
if(isset($_SESSION['zalogowany']) && $_SESSION['zalogowany']=='yes')
{
    header('location:wiadomosc.php');
}
else
{
    if(isset($_POST['login']) && isset($_POST['password']))
    {
    }
}
```

#### VI. Wyświetlenie strony z wiadomością – algorytm

---

```
if(($_POST['login']=='john') && ($_POST['password']=='123'))
{
    // 1. Ustaw jakąś wartość w $_SESSION['zalogowany'];
    // 2. Przekieruj na stronę z wiadomością;
}
else
{
    // Przekieruj na stronę z komunikatem błędu
}
```

#### VI. Wyświetlenie strony z wiadomością – kod

---

```
if(($_POST['login']=='john') && ($_POST['password']=='123'))
{
    $_SESSION['zalogowany'] = 'yes';
    header('location:wiadomosc.php');
}
else
{
    header('location:session_error.php');
}
```

## VII. Cały kod w pliku sesje.php

---

```
<?php
session_start();

if(isset($_SESSION['zalogowany']) && $_SESSION['zalogowany']=='yes')
{
    header('location:wiadomosc.php');
}
else
{
    if(isset($_POST['login']) && isset($_POST['password']))
    {
        if(($_POST['login']=='john') && ($_POST['password']=='123'))
        {
            $_SESSION['zalogowany'] = 'yes';
            header('location:wiadomosc.php');
        }
        else
        {
            header('location:session_error.php');
        }
    }
}
?>
```

## VIII. Tworzymy w menu link do pliku logout.php z napisem „Wyloguj się”

---

Plik logout.php

```
<?php
// 1. Uruchamiamy sesję
/*
    2. Sprawdzamy, czy jesteśmy zalogowani.
    Jeżeli tak:
        - zakończ sesję
        - usuń plik cookie
        - przekieruj na inną stronę
    Jeżeli nie:
        Przekieruj na stronę z logowaniem
*/
?>
```

Dane sesyjne są zapisywane w pliku cookie, dlatego też zamykając sesję powinniśmy je zniszczyć.

```
<?php
session_start();
if(isset($_SESSION['zalogowany']) && $_SESSION['zalogowany']=='yes')
{
    session_destroy();
    setcookie(session_name(),"",-1);
    header('location:sesje.php');
}
else
{
    header('location:sesje.php');
}
?>
```